End-to-End CI/CD Pipeline for a Calculator REST API

SPE Mini Project Report

Ayush Mishra
Roll No.: IMT2023129
Integrated M.Tech, Computer Science and Engineering
International Institute of Information Technology Bangalore

Repository: https://github.com/Ayush-Mishra-0018/SPE_MINI_PROJECT
Docker Hub: https://hub.docker.com/r/ayushmishra0018/calculator-devops

1 Objective and System Architecture

The objective of this project is to implement an end-to-end Continuous Integration and Continuous Deployment (CI/CD) pipeline for a Spring Boot calculator REST API. The pipeline automates source-code retrieval, compilation, testing, Docker image creation and publication, deployment, runtime health verification, and email notifications.

The system consists of GitHub for source control, Jenkins for pipeline orchestration, Maven and JUnit for building and testing, Docker for containerization, Docker Hub for image distribution, and Docker Compose for deployment. Nginx acts as the front-facing service through which the deployed API can be accessed.

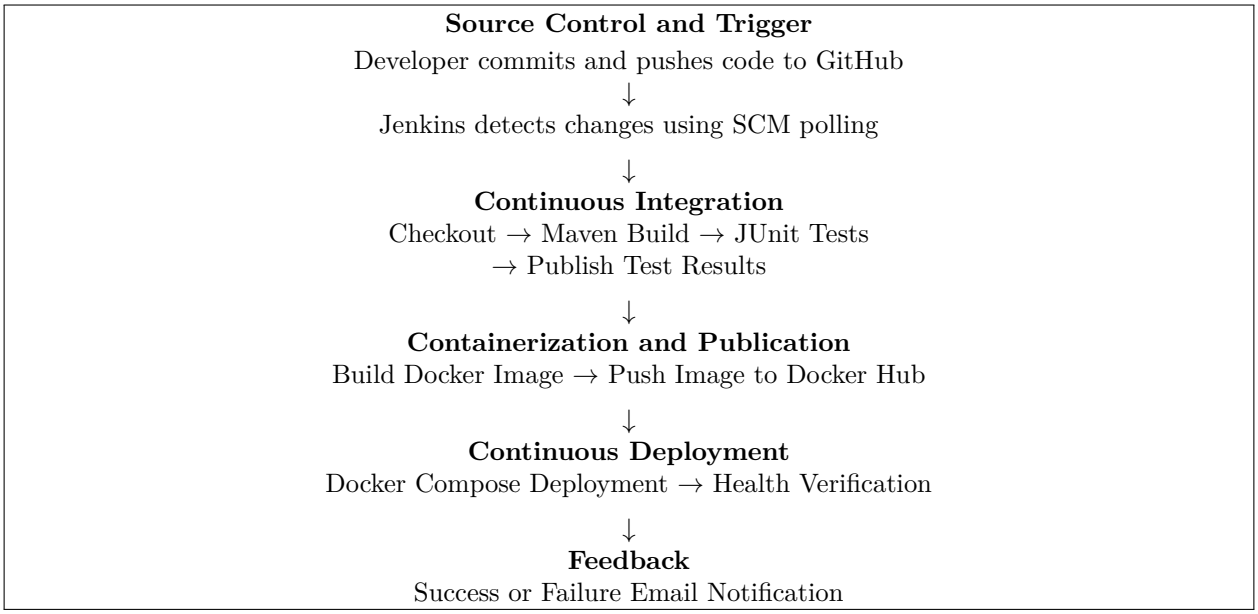


Figure 1: End-to-end CI/CD architecture and execution flow

The workflow begins when a developer pushes a change to GitHub. Jenkins periodically checks the configured repository for new commits and starts the pipeline when a change is detected. The source is checked out and built using Maven. Tests are executed before containerization so that code that fails the test stage does not proceed through the normal deployment stages.

After successful testing, Jenkins builds a Docker image and pushes it to Docker Hub using credentials stored in Jenkins. Docker Compose then deploys the application using the published image. Health checks help determine whether the API and Nginx containers are ready. Finally, Jenkins sends an email indicating the pipeline outcome.

2 Implementation and Pipeline Stages

2.1 1. Checkout

Jenkins retrieves the project source code from GitHub. This stage ensures that the build operates on the repository revision associated with the pipeline run. SCM polling provides automatic change detection without requiring a manual build after every commit.

2.2 2. Build

Maven compiles the Spring Boot application and resolves its dependencies using the project configuration in `pom.xml`. Separating the build stage makes compilation and dependency-related problems visible before testing or deployment begins.

2.3 3. Test and Publish Results

JUnit tests validate the calculator service and REST controller. The tests cover arithmetic operations, division by zero, invalid requests, and an integration-level API request. The division-by-zero case verifies that the API returns HTTP 400 with a structured JSON error rather than producing an invalid result.

Jenkins runs the tests through Maven and publishes the generated JUnit XML reports. This makes test counts and failures visible in the Jenkins interface. The verified successful run reported 11 tests, with zero failures, zero errors, and zero skipped tests.

2.4 4. Build Docker Image

After the test stage succeeds, Jenkins builds the application image using the project's Dockerfile. The Dockerfile uses a multi-stage build: the build stage uses Maven and JDK 21, while the runtime stage uses a slimmer JRE image. This avoids including the complete build toolchain in the final runtime image.

The runtime process uses a non-root user, reducing unnecessary container privileges. The image is tagged with a build-specific identifier as well as `latest`, making the deployed version easier to identify.

2.5 5. Push Image to Docker Hub

Jenkins authenticates to Docker Hub using the configured Jenkins credential, `dockerhub-creds`. It publishes the generated image to `ayushmishra0018/calculator-devops`. Credentials are kept in Jenkins rather than hard-coded in the Jenkinsfile or committed to the repository.

Publishing the image separates image creation from deployment and makes the container artifact available to the deployment environment.

2.6 6. Deploy Using Docker Compose

Docker Compose manages the calculator API and Nginx as separate services. The API runs in its own container, while Nginx provides the external entry point on host port 8081 and forwards requests to the API service over the Compose network.

The Compose configuration defines service health checks, restart policies, and service dependencies. Nginx waits for the API health check before starting dependent service operations. This improves startup coordination and makes the deployment easier to reproduce.

2.7 7. Verify Deployment

After deployment, Jenkins checks the running application through its health endpoint. The Actuator endpoint `/actuator/health` reports the application's health status. The API is also verified through Nginx to ensure that requests can reach the deployed service.

The deployed addition endpoint returned HTTP 200 with the expected result, while division by zero returned HTTP 400 with a structured error response. Both the API and Nginx containers reported healthy status in Docker Compose.

2.8 8. Email Notifications and Cleanup

Jenkins uses post-build actions to send success or failure emails. The notification includes the job name, build number, status, and a link to the console output. This gives feedback even when the pipeline is not being monitored directly.

The pipeline also performs Docker image cleanup where configured, using a best-effort image-pruning command. This helps remove unused images without making cleanup failure override the main pipeline result.

3 Challenges Faced and Solutions

1. Configuring SMTP notifications. Email delivery initially required troubleshooting the Gmail SMTP configuration. The working configuration uses `smtp.gmail.com`, port 587, TLS enabled, and SSL disabled, along with the appropriate Gmail application-password credential. After correcting the settings, successful and failed pipeline notifications were received.

2. Ensuring tests run before deployment. A CI/CD pipeline must detect application defects before publishing and deploying a new version. The stages are ordered so that testing and JUnit result publication occur before Docker image publication and deployment. A deliberately incorrect test assertion was used to exercise the failure-notification path; the subsequent correction allows a clean successful build.

3. Coordinating container startup. The API and Nginx are separate services, so container startup alone does not guarantee that the API is ready to handle requests. Health checks and Compose dependency conditions improve startup coordination. Post-deployment verification provides an additional check that the application responds correctly.

4. Managing credentials securely. Docker Hub authentication is required to publish images. Storing credentials in Jenkins and referencing them from the pipeline avoids embedding passwords or tokens in source code.

5. Keeping builds identifiable and reproducible. Build-specific image tags help distinguish versions and associate a deployed container with a Jenkins run. The `latest` tag provides a convenient reference to the most recently published image, while the build-specific tag provides clearer traceability.

4 Verification and Results

The implementation was verified across testing, publication, deployment, runtime behavior, and notification stages.

- **Automated tests:** 11 tests passed with zero failures, errors, or skipped tests in the successful run.
- **Image publication:** The Docker image and its build-specific tag were published to Docker Hub.
- **Deployment:** Docker Compose reported both the calculator API and Nginx containers as healthy.
- **API behavior:** Addition returned HTTP 200 with the expected JSON result; division by zero returned HTTP 400 with a structured JSON error.
- **Health verification:** The Actuator health endpoint returned status `UP`.
- **Notifications:** Successful and failed Jenkins runs generated email notifications.

5 Evidence Screenshots

The screenshots below provide evidence of source control, pipeline execution, testing, image publication, deployment, API behavior, health verification, and email notifications. Place the image files in a folder named `screenshots` beside this LaTeX file.

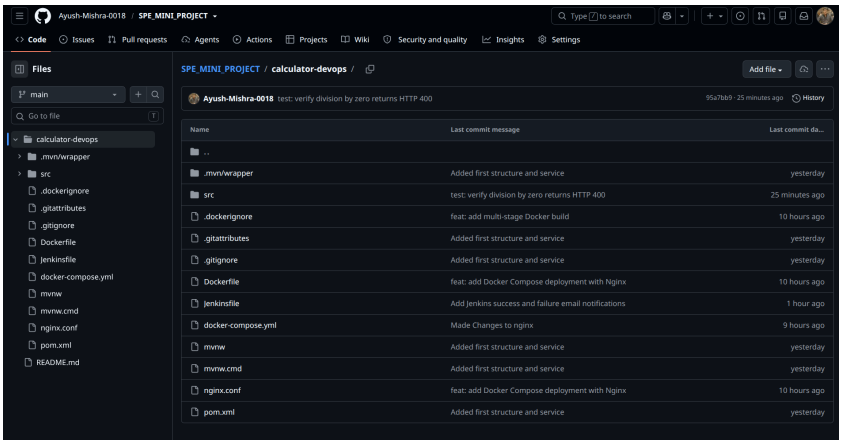


Figure 2: GitHub repository and recent commit history

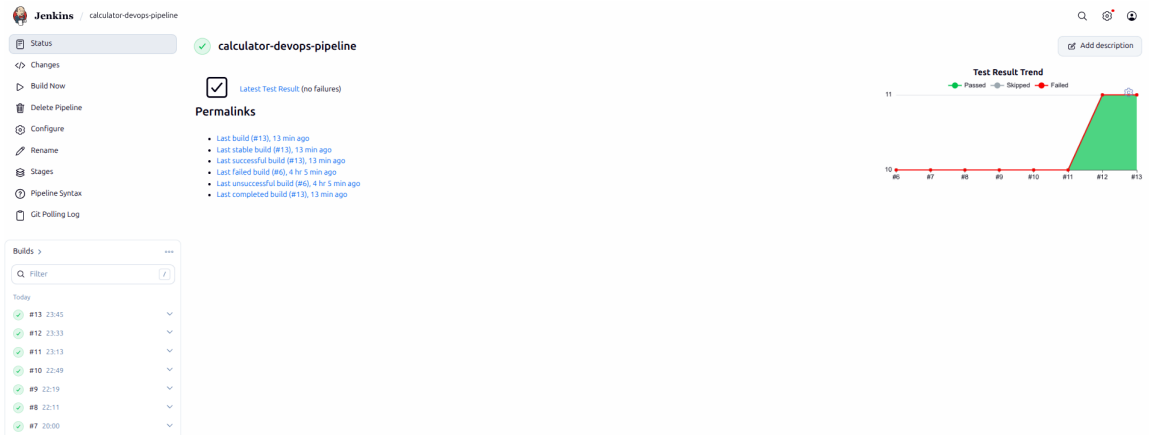


Figure 3: Jenkins successful end-to-end pipeline run

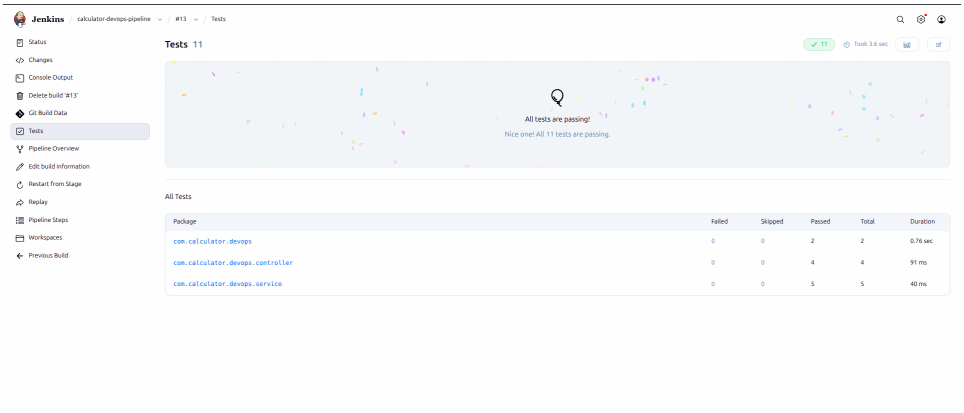


Figure 4: Published JUnit results: 11 tests and zero failures

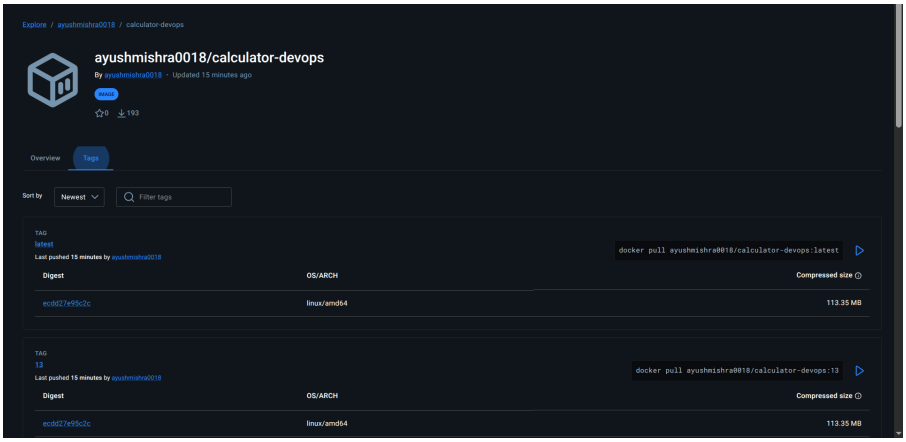


Figure 5: Docker Hub repository and published image tags

```
ayush@Swift:~/Desktop/CourseWork/Sem7/SPE/MiniProject/SPE_MINI_PROJECT/calculator-devops$ docker compose ps
NAME                                IMAGE                                    COMMAND                  SERVICE          CREATED
      STATUS                    PORTS
calculator-devops-calculator-api-1   ayushmishra0018/calculator-devops:13   "java -jar /app/app.…"   calculator-api   15 minutes
ago   Up 15 minutes (healthy)   8081/tcp
calculator-devops-nginx-1            nginx:alpine                             "/docker-entrypoint.…"   nginx            4 hours ago
      Up 4 hours (healthy)      80/tcp, 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
ayush@Swift:~/Desktop/CourseWork/Sem7/SPE/MiniProject/SPE_MINI_PROJECT/calculator-devops$
```

Figure 6: Docker Compose showing API and Nginx healthy



Figure 7: Addition endpoint returning HTTP 200 and expected JSON



Figure 8: Division by zero returning HTTP 400 and structured error



Figure 9: Actuator health endpoint returning UP



Figure 10: Successful pipeline notification in the email inbox



Figure 11: Failure notification received for a failed Jenkins build

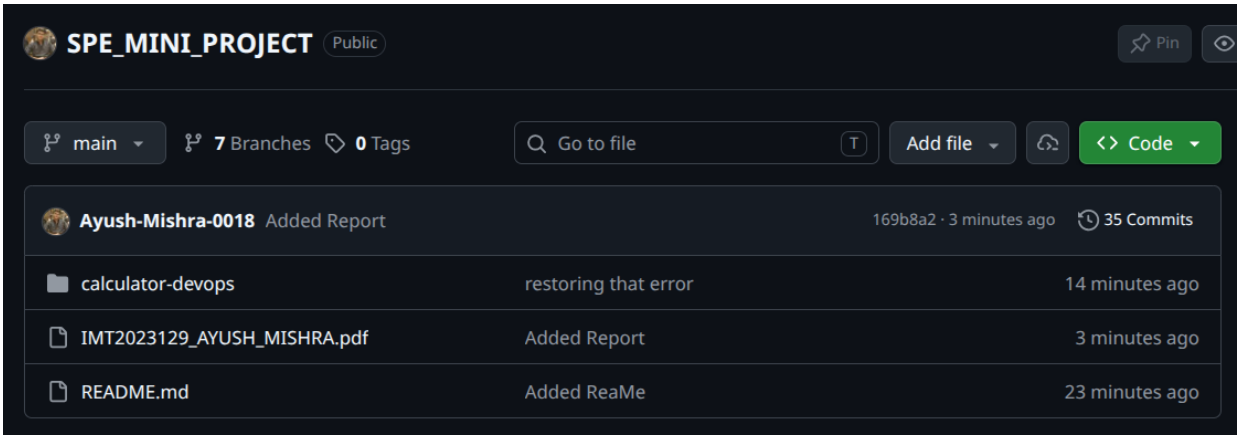


Figure 12: Branches and No. Of Commits

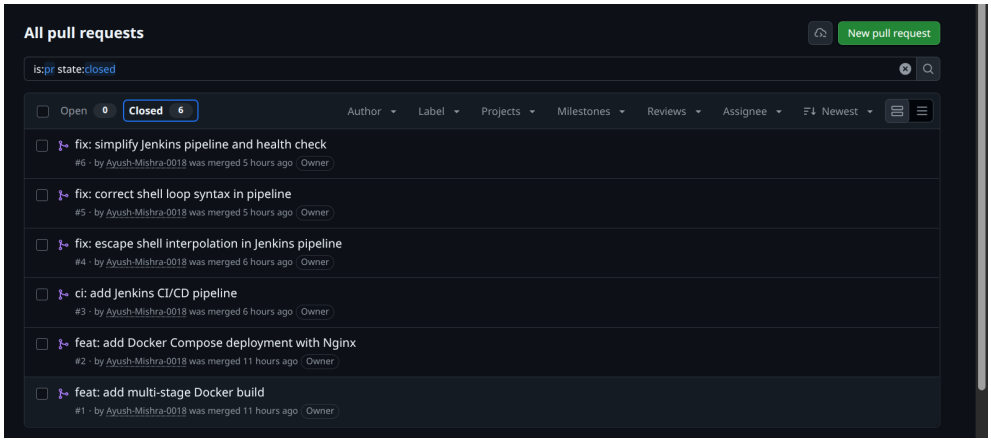


Figure 13: GitHub PR

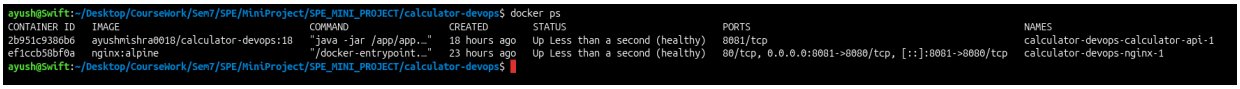


Figure 14: Docker ps

6 Conclusion

This project demonstrates an automated CI/CD workflow for a containerized Spring Boot REST API. It integrates GitHub, Jenkins, Maven, JUnit, Docker, Docker Hub, Docker Compose, Nginx, Actuator, and email notifications into a single pipeline.

Automated testing prevents ordinary pipeline execution from progressing past a failed test stage, while image publication, health checks, and notifications improve deployment traceability and observability. The controlled failure test also demonstrates the failure-notification mechanism. Together, these features provide a repeatable workflow from source-code changes to a verified running application.